

Take-Home Assignment

Receipt Parser — a small full-stack app that turns a photo into structured data.

What we're looking for

This take-home is designed to feel like real product work — the kind of thing a founding engineer at an early-stage startup might ship in a week. We care less about polish and more about **judgment**: how you scope, what you cut, how you work alongside an LLM, and how you communicate the tradeoffs you made.

You should plan to spend **3–4 focused hours** on this. If you're spending 8, you're over-engineering and we'll notice. If you're spending 1, you're under-engineering and we'll notice that too. Submit what you have at the 4-hour mark — the README is where you tell us what you'd do with more time.

A note on using LLMs

Use them. Use Cursor, Claude Code, ChatGPT, whatever your normal workflow is. We're not testing whether you can write code without one — we're testing whether you can ship well *with* one.

What we do care about is that you can explain every line you submit, and that your judgment shows up in the places the LLM can't help you: product decisions, tradeoffs, edge cases, and what to leave out.

The problem

Build a small web app that lets a user upload a photo of a receipt and get back structured data — merchant, date, line items, and total. The user should then be able to review and correct the extracted data, and save the corrected version.

Use any LLM you like for the parsing step. You pick the model, the prompt, and how you handle the response. How you make those choices is part of what we're evaluating.

Requirements

Core functionality

- A web UI where a user can upload a photo of a receipt (JPG or PNG is fine).
- A backend endpoint that accepts the image, sends it to an LLM, and returns structured data.
- A results view that shows the extracted fields: merchant name, date, line items (name + amount), and total.
- The user can edit any field inline and save the corrected version.
- Saved receipts persist somewhere (SQLite, a JSON file, anything reasonable — we're not grading your DB choice).

Technical constraints

- Use TypeScript on the backend. Frontend framework is your call — React is fine, plain HTML is fine.
- The app should run locally with a single command (e.g. `npm run dev`). Document any env vars in the README.
- Don't commit your API key. Use `.env` and add an `.env.example`.

What we're not asking for

- Authentication. Assume a single user.
- Deployment. Running locally is enough.
- A design system. Clean and usable beats pretty.
- 100% test coverage. Write tests where they'd actually catch something you care about.

Decisions we want you to make (and document)

The spec above is deliberately underspecified. A few things we've left open on purpose — these are the places your judgment shows up. You don't need to make the "right" choice; you need to make a **defensible** one and tell us why.

- **What is a line item?** Receipts have subtotals, taxes, discounts, tips. Where do you draw the line? Does it matter?
- **What happens when the LLM returns malformed output?** Retry? Fallback? Ask the user? Fail loudly?
- **How do you handle low-confidence extractions?** A blurry photo, a faded receipt, an unusual format.
- **How does the user know what to correct?** Do you surface anything beyond the raw fields?
- **Which model did you pick, and why?** Cost, latency, accuracy — you can't optimize for all three.

The correction flow is the most important part

It's tempting to spend most of your time on the LLM prompt and treat the UI as an afterthought. Resist that instinct. The reason this product would actually work is that the human catches what the LLM gets wrong — which means the correction UX is where the product lives.

We'll be paying close attention to how well that part is built.

What to submit

A GitHub repo (public, or private with access shared to the hiring contact) containing your code and a **README.md** that answers the following five questions. The README matters as much as the code.

- **What did you build?** One paragraph. No marketing language.
- **What are the biggest tradeoffs you made, and why?** Pick the two or three decisions you'd most want to defend in a PR review.

- **Where did you use an LLM, and for what?** Be specific — e.g. "used Claude for prompt iteration, wrote the API layer myself, used Cursor for test scaffolding."
- **What would you do with another week?** Shows us how you prioritize when time is finite.
- **What's one thing in this spec you'd push back on if I were your PM?** If nothing comes to mind, you haven't engaged with the problem deeply enough.

Question 5 is not a trick

We don't expect you to tear the spec apart. We do expect that after spending a few hours with the problem, you'll have noticed at least one thing that felt underspecified, wrong, or worth reconsidering. That instinct — "wait, why are we doing it this way?" — is one of the most important things a startup engineer can have.